

Platform Engineer Task — Order Supervisor on Self-Hosted Temporal

Overview

Build a POC for a long-running AI supervisor that oversees a single order from creation until completion — and deploy its orchestration backbone yourself on AWS.

Two parts:

1. **Application**: an order supervisor built on Temporal workflows.
2. **Infrastructure**: self-host Temporal and its PostgreSQL store on Kubernetes in AWS, provisioned with Terraform.

Deadline: 4 days from receiving this task.

Part 1 — Application

Core Idea

One Temporal workflow per order. The workflow supports 3 triggers for agent inference:

- workflow start,
- incoming event (delivered as a Temporal signal),
- scheduled wake-up (Temporal timer).

The agent must not run in a tight loop. It reasons, acts, updates state, decides when to sleep, and wakes later.

Agent Decision Step

The agent may be a real LLM call (any provider) or a deterministic policy / rules engine. Your choice — document why.

Wake/Sleep Behavior

- The workflow sleeps until a scheduled time or an incoming event.

- Each incoming event passes through a lightweight check (classifier or simple policy) that decides whether to wake the main agent now or stay asleep until the next scheduled wake-up.

Required Actions

Implement these 3 actions:

- `escalate_to_fulfillment_team` — flag an order issue to the fulfillment team (e.g., shipment delayed),
- `send_customer_update` — send the customer a status update or apology,
- `add_internal_note` — record an observation on the order for internal reference.

Actions don't send anything externally. Each creates an activity record stored in PostgreSQL and shown in the UI. A single activity log table for events, wake/sleep decisions, actions, instructions, and final output is fine.

Run Lifecycle Controls

From the UI/API, a user can **interrupt**, **pause**, **resume**, and **terminate** a run.

Per-Run Extra Instructions

Users can add instructions to a run after it has started, e.g. "If shipment is delayed, escalate immediately." These become part of the run context for future agent decisions.

Run Completion

The workflow must not end only because the agent decides to. Completion is system-owned:

- a terminal order event arrives (`delivered`, `cancelled`), or
- manual termination, or
- a configured max run age is reached.

On completion, write a short final summary to the activity log.

Event Generator

An endpoint, script, or UI panel that injects events into a run: `order_created`, `payment_confirmed`, `payment_failed`, `shipment_created`, `shipment_delayed`, `delivered`, `refund_requested`, `customer_message_received`.

UI (minimal)

A small Next.js UI to:

- start a run for an order,
- view active and completed runs,
- inspect a run's activity log,
- inject events,
- add extra instructions to a live run,
- interrupt, pause, resume, terminate a run.

It doesn't need to look good. The UI may run locally against your cloud API.

Part 2 — Infrastructure

Hard Requirements

1. **Temporal must be self-hosted by you on Kubernetes in AWS.** Temporal Cloud, or just pushing an image to ECS, does not count. Run the Temporal server services on a cluster you provisioned.
2. **PostgreSQL must be self-hosted by you** as Temporal's persistence store. RDS, Aurora, and Supabase are not allowed for the Temporal store — run it yourself (e.g., StatefulSet with an EBS-backed volume, or Docker on a dedicated EC2 instance). The same Postgres may serve as your app database.
3. **All AWS resources via Terraform.** `terraform apply` from a clean account should produce a working environment; `terraform destroy` should tear it all down. No console-clicked resources (documented bootstrap exceptions only).

Kubernetes Choice

- **Recommended: k3s on EC2** (single node is fine) — fast to stand up.
- **EKS is allowed.**

t3.medium instances are sufficient. A NAT gateway is not required — public subnets with strict security groups are fine for this exercise.

Terraform Must Provision

A dedicated VPC (not the default) with subnets, route tables, internet gateway; security groups with minimal rules; EC2 instances (or EKS cluster); ECR repos for your images; anything else your design needs.

What Runs on the Cluster

Temporal server (official Helm chart with your values, or your own manifests), PostgreSQL (if in-cluster), your FastAPI backend, your Temporal worker. Use standard SQL visibility — no Elasticsearch.

Monitoring (required)

Deploy a monitoring stack on the cluster (e.g., kube-prometheus-stack, or Prometheus + Grafana set up yourself):

- scrape Temporal server metrics and node/pod metrics,
- at least one Grafana dashboard showing cluster health and Temporal activity (running workflows, task latency, etc.).

Keep it lightweight — no alerting rules or long-term storage needed.

Autoscaling (required)

- Configure a HorizontalPodAutoscaler for your Temporal worker (CPU-based is fine), with metrics-server installed so it actually works.
- Demonstrate a scale-up/down event (a simple load script is enough).
- Cluster/node-level autoscaling is **not** required.

Networking & Security

- Temporal frontend (7233) reachable by your workers, not open to the internet.
 - Temporal Web UI reachable by you (port-forward, SSH tunnel, or ingress with auth) — not open to 0.0.0.0/0 unauthenticated.
 - Your API exposed via NodePort, LoadBalancer, or ingress — your choice.
 - DB credentials and API keys in Kubernetes Secrets or SSM — never hardcoded.
 - **Never commit AWS credentials or Terraform state.**
-

Deliverables

- Source code: `terraform/`, `k8s/` (or `helm/`), `backend/`, `frontend/`,
- README with exact commands from clean AWS account to working system,
- a short design note: key decisions you made (k3s vs EKS, how Postgres runs, network exposure, secrets, LLM vs rules) and why,
- a walkthrough video (10–15 min) showing: the live AWS infra, the Temporal Web UI, the Grafana dashboard, the worker HPA scaling under load, starting a run, injecting events, the agent sleeping and waking (event + timer), actions in the activity log, adding instructions to a live run, pause/resume, terminating a run, and the final summary. Record the video while your infrastructure is live.

Scope Boundaries

You don't need: HA/multi-AZ, TLS cert management, cluster/node autoscaling, alerting rules, real integrations, authentication, or polished design.

A smaller system that is fully Terraform-managed, properly networked, and reliably demoed beats a sprawling half-working one. Using AI tools is expected and encouraged — but the decisions in the result are yours to defend.